

Jetpack Compose

50 Examples from Basic to Advanced

A Comprehensive Guide with Practice Questions

Basic UI Components

Example 1: Simple Text Display (Basic)

```
@Composable
fun SimpleText() {
    Text(text = "Hello, Jetpack Compose!")
}
```

Explanation: The most basic composable function that displays text on screen.

Q: What annotation is required for a Composable function?

Example 2: Styled Text (Basic)

```
@Composable
fun StyledText() {
    Text(
        text = "Styled Text",
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        color = Color.Blue
    )
}
```

Explanation: Demonstrates how to apply styling to text including font size, weight, and color.

Q: What unit is used for font size in Compose?

Example 3: Simple Button (Basic)

```
@Composable
fun SimpleButton() {
    Button(onClick = { /* Do something */ }) {
        Text("Click Me")
    }
}
```

Explanation: A basic button with onClick handler. The lambda is called when the button is clicked.

Q: What parameter handles the click action in a Button?

Example 4: Image Display (Basic)

```
@Composable
fun DisplayImage() {
    Image(
        painter = painterResource(id = R.drawable.icon),
        contentDescription = "App Icon"
    )
}
```

Explanation: Shows how to display images from resources. contentDescription is important for accessibility.

Q: Why is contentDescription important in Image composables?

Layout Basics

Example 5: Column Layout (Basic)

```
@Composable
fun VerticalLayout() {
    Column {
        Text("First Item")
        Text("Second Item")
        Text("Third Item")
    }
}
```

Explanation: Column arranges children vertically from top to bottom.

Q: What is the default arrangement of items in a Column?

Example 6: Row Layout (Basic)

```
@Composable
fun HorizontalLayout() {
    Row {
        Text("Left")
        Text("Center")
        Text("Right")
    }
}
```

Explanation: Row arranges children horizontally from left to right.

Q: How does Row differ from Column in terms of layout direction?

Example 7: Box Layout (Basic)

```
@Composable
fun StackedLayout() {
    Box {
        Text("Background")
        Text("Foreground", modifier = Modifier.align(Alignment.Center))
    }
}
```

Explanation: Box allows overlapping children, useful for layering UI elements.

Q: What is the primary use case for Box layout?

State Management

Example 8: Remember State (Basic)

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Clicked $count times")
    }
}
```

Explanation: remember preserves state across recompositions. mutableStateOf creates observable state.

Q: What happens to state without remember during recomposition?

Modifiers

Example 9: Basic Modifiers (Basic)

```
@Composable
fun ModifiedText() {
    Text(
        text = "Modified",
        modifier = Modifier
            .padding(16.dp)
            .background(Color.LightGray)
    )
}
```

Explanation: Modifiers allow you to change the appearance and behavior of composables.

Q: What is the order of modifier application?

Example 10: Size Modifiers (Basic)

```
@Composable
fun SizedBox() {
    Box(
        modifier = Modifier
            .size(100.dp)
            .background(Color.Red)
    )
}
```

Explanation: Demonstrates how to set explicit size for composables.

Q: What is the difference between `size()` and `fillMaxSize()`?

Input Components

Example 11: TextField Input (Basic)

```
@Composable
fun TextInput() {
    var text by remember { mutableStateOf("") }
    TextField(
        value = text,
        onValueChange = { text = it },
        label = { Text("Enter text") }
    )
}
```

Explanation: TextField allows user text input with state management.

Q: Why does TextField require both `value` and `onValueChange`?

Example 12: Checkbox (Basic)

```
@Composable
fun CheckboxExample() {
    var checked by remember { mutableStateOf(false) }
    Row(verticalAlignment = Alignment.CenterVertically) {
        Checkbox(
            checked = checked,
```

```

onCheckedChange = { checked = it }
)
Text("Accept terms")
}
}

```

Explanation: Checkbox for boolean selections with state binding.

Q: How do you combine Checkbox with a clickable label?

Lists

Example 13: Simple List (Basic)

```

@Composable
fun SimpleList() {
    Column {
        listOf("Apple", "Banana", "Orange").forEach { fruit ->
            Text(fruit, modifier = Modifier.padding(8.dp))
        }
    }
}

```

Explanation: Basic way to display a list using forEach in a Column.

Q: What is the limitation of this approach for large lists?

Example 14: LazyColumn (Basic)

```

@Composable
fun EfficientList() {
    LazyColumn {
        items(100) { index ->
            Text(
                text = "Item #$index",
                modifier = Modifier.padding(8.dp)
            )
        }
    }
}

```

Explanation: LazyColumn only composes visible items, making it efficient for large lists.

Q: What is the key advantage of LazyColumn over Column?

Alignment & Arrangement

Example 15: Centered Content (Basic)

```

@Composable
fun CenteredContent() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text("Centered Text")
    }
}

```

Explanation: Shows how to center content within a container.

Q: What other alignment options are available in Box?

State Management

Example 16: State Hoisting (Intermediate)

```
@Composable
fun ParentComposable() {
    var text by remember { mutableStateOf("") }
    StatelessChild(text = text, onTextChange = { text = it })
}
@Composable
fun StatelessChild(text: String, onTextChange: (String) -> Unit) {
    TextField(value = text, onValueChange = onTextChange)
}
```

Explanation: State hoisting makes composables stateless and reusable by moving state to parent.

Q: What are the benefits of state hoisting?

Example 17: rememberSaveable (Intermediate)

```
@Composable
fun PersistentCounter() {
    var count by rememberSaveable { mutableStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Count: $count (survives config changes)")
    }
}
```

Explanation: rememberSaveable survives configuration changes like screen rotation.

Q: When should you use rememberSaveable instead of remember?

Advanced Layouts

Example 18: Custom Arrangement (Intermediate)

```
@Composable
fun SpacedColumn() {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.SpaceBetween,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Top")
        Text("Middle")
        Text("Bottom")
    }
}
```

Explanation: Custom arrangement and alignment within layouts.

Q: What is the difference between arrangement and alignment?

Example 19: Weight Modifier (Intermediate)

```
@Composable
fun WeightedRow() {
    Row(modifier = Modifier.fillMaxWidth()) {
        Box(modifier = Modifier.weight(1f).background(Color.Red))
        Box(modifier = Modifier.weight(2f).background(Color.Blue))
    }
}
```

Explanation: Weight distributes available space proportionally among children.

Q: How does `weight(2f)` compare to `weight(1f)`?

Advanced Lists

Example 20: LazyColumn with Items (Intermediate)

```
@Composable
fun ItemsList() {
    val items = listOf("A", "B", "C", "D")
    LazyColumn {
        items(items) { item ->
            Card(modifier = Modifier.padding(8.dp)) {
                Text(item, modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Explanation: Using `items()` with a list for more flexible list creation.

Q: What is the difference between `items(count)` and `items(list)`?

Example 21: LazyRow (Intermediate)

```
@Composable
fun HorizontalScroll() {
    LazyRow {
        items(20) { index ->
            Card(
                modifier = Modifier
                    .size(100.dp)
                    .padding(8.dp)
            ) {
                Text("Item $index")
            }
        }
    }
}
```

Explanation: `LazyRow` provides horizontal scrolling list.

Q: When would you use `LazyRow` instead of `LazyColumn`?

Navigation

Example 22: Basic Navigation (Intermediate)

```

@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "home") {
        composable("home") { HomeScreen(navController) }
        composable("details") { DetailsScreen() }
    }
}

@Composable
fun HomeScreen(navController: NavController) {
    Button(onClick = { navController.navigate("details") }) {
        Text("Go to Details")
    }
}

```

Explanation: Navigation component for moving between screens.

Q: What is the purpose of NavHost?

Example 23: Navigation with Arguments (Intermediate)

```

NavHost(navController, startDestination = "home") {
    composable("home") {
        Button(onClick = {
            navController.navigate("user/123")
        }) { Text("View User") }
    }
    composable(
        route = "user/{userId}",
        arguments = listOf(navArgument("userId") { type = NavType.StringType })
    ) { backStackEntry ->
        val userId = backStackEntry.arguments?.getString("userId")
        Text("User ID: $userId")
    }
}

```

Explanation: Passing arguments between navigation destinations.

Q: How do you retrieve navigation arguments in the destination?

Side Effects

Example 24: LaunchedEffect (Intermediate)

```

@Composable
fun TimerScreen() {
    var seconds by remember { mutableStateOf(0) }
    LaunchedEffect(Unit) {
        while(true) {
            delay(1000)
            seconds++
        }
    }
    Text("Seconds: $seconds")
}

```

Explanation: LaunchedEffect runs suspend functions tied to composable lifecycle.

Q: What happens when the key parameter of LaunchedEffect changes?

Example 25: DisposableEffect (Intermediate)


```

@Composable
fun LocationTracker() {
    DisposableEffect(Unit) {
        val listener = LocationListener()
        startLocationUpdates(listener)
        onDispose {
            stopLocationUpdates(listener)
        }
    }
}

```

Explanation: DisposableEffect for cleanup when composable leaves composition.

Q: When is the onDispose block called?

Material Design

Example 26: Scaffold with TopBar (Intermediate)

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ScreenWithScaffold() {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("My App") },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primary
                )
            )
        }
    ) { padding ->
        Column(modifier = Modifier.padding(padding)) {
            Text("Content here")
        }
    }
}

```

Explanation: Scaffold provides standard Material Design layout structure.

Q: Why do we need to apply the padding parameter to content?

Example 27: Card Component (Intermediate)

```

@Composable
fun InfoCard() {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Title", style = MaterialTheme typography.headlineSmall)
            Spacer(modifier = Modifier.height(8.dp))
            Text("Description text here")
        }
    }
}

```

Explanation: Card provides elevated surface for grouping content.

Q: What is the purpose of elevation in Material Design?

Theming

Example 28: Material Theme (Intermediate)

```
@Composable
fun ThemedApp() {
    MaterialTheme(
        colorScheme = lightColorScheme(
            primary = Color(0xFF6200EE),
            secondary = Color(0xFF03DAC6)
        ),
        typography = Typography(
            headlineMedium = TextStyle(fontSize = 24.sp)
        )
    ) {
        Surface(color = MaterialTheme.colorScheme.background) {
            // App content
        }
    }
}
```

Explanation: MaterialTheme provides consistent theming across the app.

Q: What are the main components of MaterialTheme?

Dialogs

Example 29: AlertDialog (Intermediate)

```
@Composable
fun ConfirmDialog() {
    var showDialog by remember { mutableStateOf(false) }
    if (showDialog) {
        AlertDialog(
            onDismissRequest = { showDialog = false },
            title = { Text("Confirm Action") },
            text = { Text("Are you sure?") },
            confirmButton = {
                TextButton(onClick = { showDialog = false }) {
                    Text("Yes")
                }
            },
            dismissButton = {
                TextButton(onClick = { showDialog = false }) {
                    Text("No")
                }
            }
        )
    }
}
```

Explanation: AlertDialog for user confirmations and important messages.

Q: How do you control the visibility of an AlertDialog?

Gestures

Example 30: Clickable Modifier (Intermediate)

```

@Composable
fun ClickableCard() {
    var clicks by remember { mutableStateOf(0) }
    Card(
        modifier = Modifier
            .padding(16.dp)
            .clickable { clicks++ }
    ) {
        Text("Clicked $clicks times", modifier = Modifier.padding(16.dp))
    }
}

```

Explanation: Making any composable clickable with the clickable modifier.

Q: What is the difference between clickable and Button?

Example 31: Swipeable Content (Intermediate)

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SwipeableItem() {
    val dismissState = rememberSwipeToDismissBoxState()
    SwipeToDismissBox(
        state = dismissState,
        backgroundColor = {
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .background(Color.Red),
                contentAlignment = Alignment.CenterEnd
            ) {
                Icon(Icons.Default.Delete, contentDescription = "Delete")
            }
        }
    ) {
        Card { Text("Swipe to delete", modifier = Modifier.padding(16.dp)) }
    }
}

```

Explanation: Implementing swipe-to-dismiss gesture.

Q: What is the purpose of backgroundColor in SwipeToDismissBox?

Animations

Example 32: Animated Visibility (Intermediate)

```

@Composable
fun FadeInOut() {
    var visible by remember { mutableStateOf(true) }
    Column {
        Button(onClick = { visible = !visible }) {
            Text("Toggle")
        }
        AnimatedVisibility(visible = visible) {
            Text("I fade in and out!")
        }
    }
}

```

Explanation: AnimatedVisibility provides enter/exit animations.

Q: What is the default animation for AnimatedVisibility?

Example 33: Animate Value (Intermediate)

```
@Composable
fun AnimatedSize() {
    var expanded by remember { mutableStateOf(false) }
    val size by animateDpAsState(
        targetValue = if (expanded) 200.dp else 100.dp,
        label = "size"
    )
    Box(
        modifier = Modifier
            .size(size)
            .background(Color.Blue)
            .clickable { expanded = !expanded }
    )
}
```

Explanation: Animating single values with animateDpAsState.

Q: What other animate*AsState functions are available?

ViewModel Integration

Example 34: Basic ViewModel (Intermediate)

```
class CounterViewModel : ViewModel() {
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count.asStateFlow()
    fun increment() {
        _count.value++
    }
}

@Composable
fun CounterScreen(viewModel: CounterViewModel = viewModel()) {
    val count by viewModel.count.collectAsState()
    Button(onClick = { viewModel.increment() }) {
        Text("Count: $count")
    }
}
```

Explanation: Integrating ViewModel with Compose for state management.

Q: Why use ViewModel instead of remember for state?

Lists with Keys

Example 35: LazyColumn with Keys (Intermediate)

```
@Composable
fun KeyedList() {
    val items = remember { mutableStateListOf("A", "B", "C") }
    LazyColumn {
        items(
            items = items,
            key = { item -> item }
        ) { item ->
            Text(item, modifier = Modifier.padding(8.dp))
        }
    }
}
```

Explanation: Using keys helps Compose identify items for efficient recomposition.

Q: Why are keys important in lists?

Custom Layouts

Example 36: Custom Layout (Advanced)

```
@Composable
fun CustomColumn(
    modifier: Modifier = Modifier,
    content: @Composable () -> Unit
) {
    Layout(
        modifier = modifier,
        content = content
    ) { measurables, constraints ->
        val placeables = measurables.map { it.measure(constraints) }
        val height = placeables.sumOf { it.height }
        layout(constraints.maxWidth, height) {
            var yPosition = 0
            placeables.forEach { placeable ->
                placeable.placeRelative(x = 0, y = yPosition)
                yPosition += placeable.height
            }
        }
    }
}
```

Explanation: Creating completely custom layouts using the Layout composable.

Q: What are the steps in the Layout lambda?

Custom Modifiers

Example 37: Custom Modifier (Advanced)

```
fun Modifier.dashedBorder(
    color: Color,
    strokeWidth: Dp = 2.dp
) = drawBehind {
    val pathEffect = PathEffect.dashPathEffect(
        floatArrayOf(10f, 10f), 0f
    )
    drawRect(
        color = color,
        style = Stroke(
            width = strokeWidth.toPx(),
            pathEffect = pathEffect
        )
    )
}

@Composable
fun DashedBox() {
    Box(
        modifier = Modifier
            .size(100.dp)
            .dashedBorder(Color.Blue)
    )
}
```

Explanation: Creating reusable custom modifiers for specialized behavior.

Q: What is the purpose of `drawBehind` vs `drawWithContent`?

Canvas & Drawing

Example 38: Canvas Drawing (Advanced)

```
@Composable
fun CustomDrawing() {
    Canvas(modifier = Modifier.size(200.dp)) {
        drawCircle(
            color = Color.Blue,
            radius = 50.dp.toPx(),
            center = Offset(100.dp.toPx(), 100.dp.toPx())
        )
        drawLine(
            color = Color.Red,
            start = Offset(0f, 0f),
            end = Offset(size.width, size.height),
            strokeWidth = 5f
        )
    }
}
```

Explanation: Using Canvas for custom drawings and graphics.

Q: What coordinate system does Canvas use?

Advanced Animations

Example 39: Transition Animation (Advanced)

```
enum class BoxState { Small, Large }
@Composable
fun TransitionAnimation() {
    var currentState by remember { mutableStateOf(BoxState.Small) }
    val transition = updateTransition(currentState, label = "box")
    val size by transition.animateDp(label = "size") { state ->
        when(state) {
            BoxState.Small -> 50.dp
            BoxState.Large -> 150.dp
        }
    }
    val color by transition.animateColor(label = "color") { state ->
        when(state) {
            BoxState.Small -> Color.Blue
            BoxState.Large -> Color.Red
        }
    }
    Box(
        modifier = Modifier
            .size(size)
            .background(color)
            .clickable {
                currentState = if (currentState == BoxState.Small)
                    BoxState.Large else BoxState.Small
            }
    )
}
```

Explanation: Coordinating multiple animations with `updateTransition`.

Q: What advantage does `updateTransition` provide over individual animations?

Example 40: Infinite Animation (Advanced)

```
@Composable
fun PulsingCircle() {
    val infiniteTransition = rememberInfiniteTransition(label = "pulse")
    val scale by infiniteTransition.animateFloat(
        initialValue = 1f,
        targetValue = 1.5f,
        animationSpec = infiniteRepeatable(
            animation = tween(1000),
            repeatMode = RepeatMode.Reverse
        ),
        label = "scale"
    )
    Box(
        modifier = Modifier
            .size(100.dp)
            .scale(scale)
            .background(Color.Red, CircleShape)
    )
}
```

Explanation: Creating infinite repeating animations.

Q: What is the difference between `RepeatMode.Reverse` and `RepeatMode.Restart`?

Gestures

Example 41: Drag Gesture (Advanced)

```
@Composable
fun DraggableBox() {
    var offsetX by remember { mutableStateOf(0f) }
    var offsetY by remember { mutableStateOf(0f) }
    Box(
        modifier = Modifier
            .offset { IntOffset(offsetX.roundToInt(), offsetY.roundToInt()) }
            .size(100.dp)
            .background(Color.Blue)
            .pointerInput(Unit) {
                detectDragGestures { change, dragAmount ->
                    change.consume()
                    offsetX += dragAmount.x
                    offsetY += dragAmount.y
                }
            }
    )
}
```

Explanation: Implementing drag gestures with `pointerInput`.

Q: Why do we call `change.consume()`?

CompositionLocal

Example 42: Custom CompositionLocal (Advanced)

```
data class AppSettings(val theme: String, val language: String)
val LocalAppSettings = compositionLocalOf {
    AppSettings("Light", "EN")
}
@Composable
fun App() {
    CompositionLocalProvider(
        LocalAppSettings provides AppSettings("Dark", "ES")
    ) {
        ChildComposable()
    }
}
@Composable
fun ChildComposable() {
    val settings = LocalAppSettings.current
    Text("Theme: ${settings.theme}")
}
```

Explanation: CompositionLocal provides implicit data down the composition tree.

Q: When should you use CompositionLocal vs parameter passing?

Derived State

Example 43: derivedStateOf (Advanced)

```
@Composable
fun SearchableList() {
    var query by remember { mutableStateOf("") }
    val items = remember { List(1000) { "Item $it" } }
    val filteredItems by remember {
        derivedStateOf {
            items.filter { it.contains(query, ignoreCase = true) }
        }
    }
    Column {
        TextField(value = query, onValueChange = { query = it })
        LazyColumn {
            items(filteredItems) { item ->
                Text(item)
            }
        }
    }
}
```

Explanation: derivedStateOf optimizes calculations that depend on state.

Q: How does derivedStateOf improve performance?

Side Effects

Example 44: SideEffect (Advanced)

```
@Composable
fun AnalyticsTracker(screenName: String) {
    SideEffect {
        // This runs after every successful recomposition
        analytics.logScreenView(screenName)
    }
}
```



```
}
```

Explanation: SideEffect runs after every successful recomposition for non-Compose state updates.

Q: When should you use SideEffect vs LaunchedEffect?

Performance

Example 45: Stable Classes (Advanced)

```
@Immutable
data class User(val id: Int, val name: String)
@Stable
class UserRepository {
    private val _users = mutableStateListOf<User>()
    val users: List<User> = _users
    fun addUser(user: User) {
        _users.add(user)
    }
}
@Composable
fun UserList(repository: UserRepository) {
    LazyColumn {
        items(repository.users) { user ->
            UserItem(user)
        }
    }
}
@Composable
fun UserItem(user: User) {
    Text(user.name)
}
```

Explanation: @Stable and @Immutable help Compose optimize recomposition.

Q: What is the difference between @Stable and @Immutable?

Example 46: Key for Recomposition (Advanced)

```
@Composable
fun OptimizedList(items: List<Item>) {
    LazyColumn {
        items(
            items = items,
            key = { it.id }
        ) { item ->
            key(item.lastModified) {
                ItemCard(item)
            }
        }
    }
}
```

Explanation: Using key() to force recomposition when specific values change.

Q: How does key() differ from the key parameter in items()?

Custom Remember

Example 47: Custom Remember Function (Advanced)

```
@Composable
fun <T> rememberMutableStateListOf(vararg elements: T): SnapshotStateList<T> {
    return remember { mutableStateListOf(*elements) }
}

@Composable
fun <T> rememberFlowWithLifecycle(
    flow: Flow<T>,
    lifecycle: Lifecycle = LocalLifecycleOwner.current.lifecycle
): Flow<T> {
    return remember(flow, lifecycle) {
        flow.flowWithLifecycle(lifecycle)
    }
}
```

Explanation: Creating custom remember functions for reusable state patterns.

Q: Why include dependencies in remember parameters?

Modifier Composition

Example 48: Complex Modifier Chain (Advanced)

```
@Composable
fun AdvancedModifiers() {
    var scale by remember { mutableStateOf(1f) }
    Box(
        modifier = Modifier
            .size(200.dp)
            .clip(RoundedCornerShape(16.dp))
            .background(
                brush = Brush.verticalGradient(
                    colors = listOf(Color.Blue, Color.Cyan)
                )
            )
            .border(2.dp, Color.White, RoundedCornerShape(16.dp))
            .scale(scale)
            .pointerInput(Unit) {
                detectTapGestures(
                    onPress = { scale = 0.9f },
                    onTap = { scale = 1f }
                )
            }
            .semantics { contentDescription = "Pressable gradient box" }
    )
}
```

Explanation: Complex modifier chains with multiple effects and interactions.

Q: Why does modifier order matter?

Advanced State

Example 49: SnapshotFlow (Advanced)

```
@Composable
fun ScrollObserver() {
    val listState = rememberLazyListState()
    LaunchedEffect(listState) {
        snapshotFlow { listState.firstVisibleItemIndex }
    }
}
```

```

        .distinctUntilChanged()
        .collect { index ->
println("First visible item: $index")
        }
    }
    LazyColumn(state = listState) {
        items(100) { index ->
            Text("Item $index")
        }
    }
}

```

Explanation: Converting Compose state to Flow using snapshotFlow.

Q: What is the advantage of snapshotFlow over direct state observation?

Interop

Example 50: AndroidView Integration (Advanced)

```

@Composable
fun WebViewComposable(url: String) {
    AndroidView(
        factory = { context ->
            WebView(context).apply {
                settings.javaScriptEnabled = true
                webViewClient = WebViewClient()
                loadUrl(url)
            }
        },
        update = { webView ->
            webView.loadUrl(url)
        }
    )
}

@Composable
fun MapViewComposable() {
    val mapView = rememberMapViewWithLifecycle()
    AndroidView({ mapView }) { view ->
        // Update map
    }
}

```

Explanation: Integrating Android Views into Compose with AndroidView.

Q: When should you use factory vs update in AndroidView?

Practice Questions

1. Explain the difference between `remember` and `rememberSaveable`. When would you use each?
2. What is state hoisting and why is it important in Jetpack Compose?
3. How does `LazyColumn` improve performance compared to a regular `Column` with a large list?
4. Describe the Compose recomposition process. What triggers it?
5. What are the three phases of Compose layout (Composition, Layout, Drawing)?
6. Explain modifier order and provide an example where order matters.
7. What is the purpose of `CompositionLocal`? Give an example use case.
8. How do you prevent unnecessary recompositions in Compose?
9. What is the difference between `LaunchedEffect` and `DisposableEffect`?
10. Explain how to integrate a `ViewModel` with Compose and manage state properly.
11. What are `stable` and `immutable` annotations? How do they affect performance?
12. How would you implement a custom layout in Jetpack Compose?
13. Describe the purpose of `derivedStateOf` and when to use it.
14. How do you handle navigation with arguments in Jetpack Compose?
15. What are the best practices for organizing composable functions?
16. Explain the difference between `Modifier.clickable` and `Button`.
17. How do you create custom modifiers in Compose?
18. What is the role of `Scaffold` in Material Design layouts?
19. How would you implement a drag gesture in Compose?
20. Describe how to integrate traditional Android Views with Compose using `AndroidView`.